Battle Hunter, Recompiled

How a PlayStation game became a real PC program — explained for everyone



A plain-English case study of static recompilation

PlayStation 1 · Battle Hunter (SLUS-01335) · October 2026

Built with the open-source psxrecomp framework on an ordinary Linux PC. The game disc and its music remain the property of their creators; this project translates a personally-owned copy.

1. The short version

Old video games are programs written for a specific console's chip. A PlayStation 1 game is a bundle of instructions for a MIPS processor — the 'brain' Sony put in the machine in 1994. A modern PC's brain speaks a completely different language (x86-64), which is why old games normally need an emulator: a program that pretends to be the whole console and translates the game's instructions on the fly, over and over, forever.

This project took the other road, called static recompilation: the game's 688 KB program was machine-translated — once, ahead of time — into ordinary C code, then compiled by a regular PC compiler into a normal executable. What runs now is the game's own logic, executing directly as PC instructions. No emulator is in the loop.

- The squeezed disc file that was given to me was unpacked, and its missing track layout was rebuilt by forensic analysis of the raw bytes.
- The game program became 4.75 million lines of C (1,873 functions), compiled into one 37 MB executable.
- The console's startup software (BIOS) is also recompiled — a free, open one (OpenBIOS), so nothing copyrighted is needed from Sony.
- The parts around the CPU — graphics, sound, the disc drive — were rebuilt as normal PC software doing the same jobs, like a printer driver replaces a printer's brain.
- Verified by real play: boot logos, menus, battles, music, saving, loading — and one honest game over.

Three ways to play a 25-year-old game

Way	How it works	The catch
Original console	Real hardware runs the real disc.	Needs aging hardware and a CRT-era setup.
Emulator	A program impersonates the console and translates every instruction while you play.	Works well, but the game never becomes PC software; the emulator is a permanent middleman.
Recompilation (this project)	Translate the game once into PC instructions, then just run it.	One-time reverse-engineering effort per game — the result is simply a program.

In plain English:
an emulator is a live interpreter whispering every sentence as you read a foreign book. Recompilation is translating the whole book once and printing it in English.

2. What we started with: one mysterious file

The whole task began with a single 81 MB file named 'BATTLE HUNTER.iso.ecm'. Two problems hid inside that name:

- '.ecm' means the disc image had been squeezed by ECM, an old tool that deletes a CD's built-in error-correction bytes (they can be recomputed from the data, so storing them is wasted space). Fine for sharing in 2008 — unusable directly today.

- A PlayStation disc is mixed-media: one data track plus several music tracks. Players need a small text 'table of contents' (a cue sheet) saying where each track begins. None came with the file.
- Worse, the machine had no ECM-unpacking software installed and no administrator rights to add any.

So the first job was to build the unsqueezer from its original source code — 376 lines of C written in 2002 by Neill Corlett. It compiled in under a second.

Result of step one:
86,003,232 bytes of raw disc — exactly 36,566 'sectors'. A sector is a CD's unit of storage (about 2.3 KB); 75 of them pass by per second, so this disc holds 8 minutes 7 seconds of material.

3. Playing disc detective: rebuilding the track list

With no cue sheet, the disc's layout had to be reconstructed from the bytes themselves. Fortunately, CDs leave fingerprints:

- Data sectors announce themselves: every one begins with the same 12-byte 'sync' pattern plus a header — like every page of a document carrying the same letterhead.
- Music sectors carry no letterhead: they are pure audio numbers, and loud passages look like noise.
- Scanning all 36,566 sectors found: the first 4,180 (about 56 seconds) are data; the remaining 32,386 (7.2 minutes) are music.

But where does each song start? Commercial CDs leave a two-second silent 'pregap' in front of every track. Searching for exactly 150-sector silences found four of them — at 0:55, 2:29, 3:59 and 5:28. Four gaps, four songs.

Then a lucky cross-check: the disc's data track contains a vestigial file list left over from mastering. Its entries point at exactly those music starts — a 'file' named HUNTER2.DA at sector 4330 (the start of song 2), and so on. The 'files' were actually the songs.

Conclusion: five tracks — one data, four music. A 19-line cue sheet was written. When the correct songs later played at the correct moments in testing, that was final proof the reconstruction was right.



The main menu, running as native PC code. (Screenshot captured through the debug port during testing.)

4. The translation: MIPS to C to machine code

The translation itself was done by psxrecomp, an open-source recompiler framework for PlayStation games — the same family of tools behind recent native PC ports of Tomba!, Ape Escape and Mega Man X4–X6. Its pipeline is beautifully simple to describe:

```
PlayStation disc ──> unpack game program (MIPS machine code)
                     ──> analyse: find the 1,873 functions
                     ──> generate: one C file per group of functions
                     ──> compile with GCC (the PC's own compiler)
                     ──> a single executable: BattleHunter_Recompiled
```

The game's program (SLUS_013.35, 704,512 bytes of MIPS instructions) became 4.75 million lines of C in 109 files. A first pass found 717 entry points by scanning the game's jump instructions; the tool discovered the remaining functions by following the code's own footprints.

Every console also needs built-in startup software — the BIOS. Instead of Sony's copyrighted one, the free OpenBIOS was used, and it was recompiled the same way. So even the 'kernel' that boots the game is native code.

Finally, GCC compiled everything into one 37 MB executable. On this machine, the entire build took about two and a half minutes.

The languages involved:

the game's MIPS machine code became C (generated automatically by machine, not by hand), which GCC turned into x86-64 machine code. Around it: the framework's C/C++ runtime that stands in for the console's other chips, plus small Python and C tools written for the disc forensics and testing.

5. Making sure it really works

A recompiled game is only trustworthy if it behaves like the original, so testing was systematic — and at one point, genuinely funny.

Lights, camera, debug port

The game was first booted 'headless': no window, everything watched through a built-in debug connection. I sent commands to take screenshots and to press virtual buttons, and the boot logos, title screen and music all appeared on cue. Over thirteen million calls into the translated game code happened during testing — every single one reached a pre-translated function. None needed a fallback.

The mystery of the too-fast game

During early testing the game ran unthrottled, about seventeen times real speed. Press Start and… black screen, then a quiet retreat back to the title. Diagnosis: the game waits on disc loads with a timeout counted in screen refreshes. At 17× speed, the disc (which is paced by the real clock) could never keep up with the hyperactive console, so the game gave up. Re-run at exactly 60 frames per second, everything worked. Lesson learned: soak-test at the speed the game was designed for.

The controller that worked… in the menus only

An 8BitDo wireless controller worked perfectly in the launcher, then went dead in-game while the keyboard

worked fine. The cause was one default in a settings file: player 1 was assigned to 'keyboard' rather than the gamepad. One line later, every button press on the pad could be watched flowing into the game through the debug port.



A story scene during soak testing — input, music and scene transitions all verified.

The human test

The final exam was a real play session: menus, dialogue, hunts, battles — ending in a proper game over — plus saving to and loading from the memory card. If a game can defeat you fairly and remember it happened, it works.

6. The last mile: code that streams off the disc

Some games don't keep all their program in the main executable: they stream extra code modules from the disc into spare memory while playing — a bit like levels being loaded on demand. You cannot pre-translate code you don't have yet.

The framework's answer is a two-layer trick. While such code runs the first time, a tiny backup interpreter (a miniature emulator for just those fragments) keeps everything correct — and the system secretly records ('captures') the code the moment it loads. After the play session, the three captured modules were fed back through the same translation pipeline and compiled into native plug-in libraries, each guarded by a byte-check: if the game ever loads different bytes there, the plug-in stands down and the safe path takes over again.

Before / after the overlay work:
the interpreter had executed 4,589,668 instructions. After: 24. Everything else — every function in the game, the BIOS, and the streamed modules — now runs as direct PC code.

7. By the numbers

Measure	Value
Disc image (squeezed .ecm / unpacked .bin)	81,280,451 bytes → 86,003,232 bytes (36,566 sectors)

Disc layout reconstructed	5 tracks: 1 data (56 s) + 4 music (7.2 min total)
Game program (PlayStation MIPS)	704,512 bytes
Generated C code	4,751,646 lines in 109 files — 1,873 functions
Final executable (native x86-64)	37 MB, built by GCC in ~2.5 minutes
Translated-code calls during testing	13,602,553 — with 0 misses
Interpreter use, before → after overlays	4,589,668 instructions → 24 instructions
Disc-streamed modules compiled natively	3 modules → 5 native plug-ins, 6 functions registered
Verified in play	Boot, menus, battles, game over, save & load, music, controller

8. How to play it on this machine

- Easy way: run the script ~/Documents/BHRecomp/play-battle-hunter.sh
- Or launch BattleHunterRecomp/build-release/BattleHunter_Recompiled directly.
- On first run the launcher may ask for the disc image — pick disc/BattleHunter.cue.
- Controller: your 8BitDo pad is already configured as player 1. Keyboard: arrows = D-pad, X = Cross, S = Circle, Z = Square, A = Triangle, Enter = Start.
- Saves live in BattleHunterRecomp/saves/ as memory-card files — the original disc files are never modified.

9. Glossary

Term	Plain-English meaning
Emulation	A program pretending to be the whole console, translating the game as it runs.
Static recompilation	Translating a game's program once, ahead of time, into PC instructions.
MIPS / x86-64	The PlayStation's CPU language / the modern PC's CPU language.
C	A general-purpose programming language PCs are built with; the intermediate stop for the translated code.
Compiler (GCC)	The program that turns C into machine code; the same one that builds Linux software.
Sector	A CD's unit of storage, about 2.3 KB; 75 pass by per second.
Cue sheet (.cue)	A text file listing a disc's tracks and where each begins.
BIOS	The console's built-in startup software. Here: OpenBIOS, also recompiled.
Overlay	Extra game code streamed from the disc into memory at runtime.
Interpreter	The small backup that runs untranslated code; now nearly unused (24 instructions).
Native code	Instructions your CPU executes directly — no translation happening.

10. Tools and credits

- psxrecomp — the recompiler framework and runtime, by mstan and the RetroPortingToolkit community (github.com/mstan/psxrecomp).
- OpenBIOS — the free PlayStation BIOS, from the PCSX-Redux project.
- unecm — the ECM disc decoder, by Neill Corlett (cmdpack, 2002).
- recomp-ui — the launcher interface. GCC, SDL3, Dear ImGui and many other open-source pieces underneath.
- Project files: ~/Documents/BHRecomp/BattleHunterRecomp (git-tracked; status marked 'playable').

Battle Hunter was developed by its original creators and published for the PlayStation around the year 2000. All trademarks and the game itself remain the property of their respective owners. This translation was made for personal use from a personally-owned copy of the game.